



## **Informe Técnico: Implementación de un Sistema de Atención mediante una Cola (MyDeque)**

ICC708-1: PROGRAMACION AVANZADA

**Nombre:** Bárbara Constanza Sanhueza Bombín.

**Carrera:** Ingeniería informática.

**Fecha:** 24-06-2026.

**Docente:** Profesor Ricardo Gacitúa B.



## Índice

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Descripción del Problema y Modelo de Solución</b>  | <b>3</b> |
| Arquitectura de la Estructura Doblemente Enlazada     | 3        |
| Manejo de Casos Borde y Validaciones                  | 4        |
| <b>Estrategia de Verificación y Pruebas Unitarias</b> | <b>4</b> |
| <b>Conclusión</b>                                     | <b>4</b> |

## Descripción del Problema y Modelo de Solución

El presente proyecto aborda el diseño y desarrollo de un sistema automatizado de atención de público. Este tipo de entornos exige la gestión de flujos de usuarios con diferentes niveles de prioridad, y dinámicas cambiantes; representados por cuatro operaciones esenciales:

- **Cientes Normales:** Se incorporan de manera secuencial al final de la fila de espera.
- **Cientes Prioritarios:** Se ingresan directamente al principio de la fila para recibir atención preferencial inmediata.
- **Atención de Clientes:** Se procesa y remueve al usuario ubicado en el primer lugar de la fila.
- **Cancelaciones de Turno:** Se remueve al último usuario de la fila en caso de que decida abandonar la espera de forma voluntaria.

Para dar soporte óptimo a estos requerimientos sin incurrir en penalizaciones de rendimiento, se implementó una estructura de datos lineal lineal denominada Cola Doblemente Terminada o Deque (MyDeque). Esta estructura permite realizar inserciones y remociones en ambos extremos de la secuencia en tiempo constante, adaptándose exactamente a las necesidades del sistema de atención.

## Arquitectura de la Estructura Doblemente Enlazada

La solución se ha diseñado utilizando punteros y memoria dinámica para evitar las restricciones de tamaño fijo asociadas a los arreglos tradicionales. La arquitectura se compone de las siguientes clases interconectadas:

- **Clase Nodo:** Representa la unidad fundamental de almacenamiento de la estructura. Contiene el dato genérico (T) y dos referencias de enlace bidireccional: un puntero al nodo siguiente y un puntero al nodo anterior.
- **Clase MyDeque:** Implementa la lógica de control del Deque mediante tres variables de estado: un puntero al nodo inicio, un puntero al nodo fin y un contador entero tamaño que registra la cantidad de elementos activos en la estructura.

### Mecánica de Operación de Punteros

El Deque gestiona sus enlaces de manera simétrica en sus dos extremos:

- **Inserción (*addFirst* / *addLast*):** Cuando la estructura está vacía, los punteros inicio y fin se dirigen al nuevo nodo de forma simultánea. Si ya existen datos, los nuevos elementos enlazan su puntero correspondiente con el nodo extremo actual, y la estructura actualiza la referencia inversa (*inicio.anterior* o *fin.siguiente* según corresponda) para mantener la coherencia y transitabilidad en ambos sentidos.
- **Remoción (*removeFirst* / *removeLast*):** El sistema extrae el valor del extremo solicitado, desplaza el puntero principal (*inicio* hacia el siguiente, o *fin* hacia el anterior) y rompe el enlace residual asignándole el valor null.

## Manejo de Casos Borde y Validaciones

Para construir un sistema más confiable y siempre disponible, añadimos controles estrictos que detectan y manejan cualquier error inesperado al momento de ejecutarse.

- *Control de Subflujo (Estructura Vacía)*: Si se solicita una remoción en una cola sin elementos, los métodos detienen la operación de forma segura, imprimen una alerta y retornan null, evitando interrupciones por excepciones de puntero nulo.
- *Sanitización de Instrucciones*: El procesador de eventos descarta cadenas vacías y verifica que comandos como NORMAL o PRIORITARIO incluyan el nombre del cliente. Esto previene que el sistema colapse por índices fuera de rango ante argumentos incompletos.

## Estrategia de Verificación y Pruebas Unitarias

A través del framework JUnit 5, se estructuró una suite automatizada de pruebas enfocada en certificar la fiabilidad del componente bajo tres enfoques:

- *Inserciones Extremas*: Validación del posicionamiento correcto de los datos y del incremento preciso del contador de tamaño tras operaciones consecutivas en ambos extremos.
- *Consistencia Bidireccional*: Ejecución de flujos combinados para asegurar que los punteros inversos (anterior) mantengan la coherencia y permitan la transitabilidad completa de la lista en ambos sentidos.
- *Ciclos de Transición de Estado*: Pruebas de llenado y vaciado sucesivo que confirman que todas las referencias internas regresan con precisión a null sin dejar objetos huérfanos en la memoria.

## Conclusión

El uso de una Lista Doblemente Enlazada garantiza que el Sistema de Atención funcione de forma instantánea al agregar o quitar elementos en los extremos. La separación clara de sus componentes hace que el código sea limpio y fácil de mantener, mientras que las pruebas automatizadas aseguran que el sistema sea completamente confiable y libre de errores.